

PRUEBA TÉCNICA

DESARROLLADOR/A ANDROID

Tecnologías Principales: Java, RxJava

Arquitectura Sugerida: MVVM o MVP

Inyección de Dependencias: Dagger 2

Información del Candidato/a:

- **Nombre Completo:** Sergio Enrique Vargas Pedraza (colombia00028@gmail.com)
- **Fecha:** 2025/05/19

Instrucciones Generales:

- El objetivo de esta prueba es evaluar tu capacidad para desarrollar una aplicación Android funcional, modular y bien estructurada, utilizando las tecnologías y patrones especificados.
- Se espera que implementes las funcionalidades descritas a continuación, prestando especial atención a la calidad del código, la aplicación del patrón de arquitectura elegido (MVVM o MVP), el uso correcto de Dagger 2 para la inyección de dependencias y el manejo eficiente de operaciones asíncronas con RxJava.
- El diseño de la interfaz de usuario debe seguir la guía de estilo proporcionada, enfocándose en la paleta de colores (principalmente tonos de azul) y un estilo de iconografía limpio y moderno.
- Deberás configurar un pipeline de CI/CD utilizando GitHub Actions para compilar, ejecutar pruebas y desplegar la aplicación en Firebase Distribution. **La generación del APK para Firebase Distribution debe hacerse en el proyecto de Firebase que tú crees, el mismo que se utiliza para Firestore. Los testers descargarán el APK desde allí.**
- La solución debe ser entregada como un proyecto de Android Studio funcional y el enlace al repositorio de GitHub donde se encuentre el código y la configuración de GitHub Actions. **Asegúrate de asociar al repositorio de GitHub a los usuarios/correos especificados para su revisión.**
- Se debe implementar Volley como petición http, una librería para QR scanning, una librería para mapas, etc.), pero justifica tu elección si no son las más comunes.
- Maneja adecuadamente los posibles errores (red, base de datos, validaciones, etc.) utilizando los mecanismos de RxJava.

- Documenta cualquier suposición importante que hayas hecho durante la implementación.
- Tiempo estimado para la prueba: la prueba deberá entregarse el día lunes 19 de mayo a las 6:00 pm enviando un correo adjuntando los links de github, de firebase

Descripción del Ejercicio:

Deberás desarrollar una aplicación Android con dos pantallas principales: una pantalla de inicio de sesión y una pantalla principal con funcionalidades diversas.

Requisitos Detallados:

1. Pantalla de Inicio de Sesión (Login):

- Interfaz de usuario (`layout.xml`) con campos de texto para usuario y contraseña y un botón "Iniciar Sesión".
- **Estilo de Diseño:** Aplica la paleta de colores basada en tonos de azul observada, texto principal en negro o gris oscuro, y considera un color de acento (ej. naranja/rojo para advertencias). Los campos de texto y botones deben tener un estilo limpio, posiblemente con bordes redondeados o líneas inferiores distintivas, y los botones de acción primaria deben tener un fondo de color azul sólido con texto claro (blanco). Utiliza una tipografía clara y legible.
- Al presionar "Iniciar Sesión", la aplicación debe validar las credenciales (usuario y contraseña) contra una colección llamada `Usuarios` en un proyecto de Firebase Firestore .
- Si las credenciales no son correctas, mostrar un mensaje de error (`Toast` o `Snackbar`) indicando "Usuario no existe o credenciales incorrectas".
- Si las credenciales son correctas, navegar a la pantalla principal.

2. Pantalla Principal:

- Interfaz de usuario (`layout.xml`) que contenga:
 - Un icono de "Cerrar Sesión" en la parte superior derecha.
 - Un texto debajo del icono que diga "Leer Información".
 - Un campo de texto centrado horizontalmente.
 - Un icono de "Cámara" junto al campo de texto.
 - Un `RecyclerView` centrado en la pantalla que mostrará una lista de elementos.
- **Estilo de Diseño:** Continúa aplicando la paleta de colores azules. La iconografía debe ser coherente con un estilo limpio y moderno, preferiblemente iconos de línea. Los elementos interactivos (iconos, botones) deben ser claramente distinguibles.
- **Funcionalidad "Cerrar Sesión":**
 - Al presionar el icono de "Cerrar Sesión", la aplicación debe:
 - Eliminar la data asociada al dispositivo actual (basado en algún identificador único del dispositivo o un identificador de sesión si

lo implementas) de una colección de Firebase Firestore llamada **backup**.

- Eliminar todos los datos de una tabla local SQLite llamada **backup_local**.
- Navegar de regreso a la pantalla de inicio de sesión.
- Todas estas operaciones (Firebase delete, SQLite delete) deben manejarse de forma asíncrona utilizando RxJava y asegurando que se completen antes de regresar a la pantalla de login.

3. Funcionalidad de Cámara y QR Scanning:

- Al presionar el icono de "Cámara", se debe mostrar una vista de cámara en la parte superior de la pantalla, desplazando hacia abajo el campo de texto, el icono de cámara y el **RecyclerView**.
- La vista de cámara debe incluir un icono para cerrarla (siguiendo el estilo de iconografía).
- Al presionar el icono de cerrar en la vista de cámara, esta debe ocultarse y los elementos del layout deben regresar a su posición original.
- La cámara solo debe estar configurada para leer códigos QR.
- Cuando se detecta y lee un código QR, se debe capturar el dato.

4. Procesamiento del Código QR y Comunicación con API:

- El dato capturado del código QR (que se espera sea un String en Base64) debe ser enviado a través de una petición HTTP POST al endpoint:
<https://noderedtest.coordinadora.com/api/v1/validar>.
- Esta petición debe enviar un JSON con la siguiente estructura:

```
{"data":"[contenido_del_qr_en_base64]"}
```

Donde **[contenido_del_qr_en_base64]** es el String leído del código QR.

El servicio validará si el contenido Base64 decodificado tiene la estructura esperada (**etiqueta1d:...-latitud:...-longitud:...-observacion:...**).

Respuestas del Servicio:

- **Estructura Correcta:**

JSON

```
{  
  "Correcto": "estructura Correcta",  
  "data": "etiqueta1d:720024556544001-latitud:6.74444-longitud:-75.45454-  
observacion:prueba"  
}
```

Estructura Incorrecta:

JSON

```
{  
  "Correcto": "estructura Incorrecta",
```

```
"data": "etiqueta1d:720024556544001-latitud:6.74444-longitud:-75.45454,observacion:prueba" // Nota la coma en lugar del guion
}
```

-

- La comunicación con el servicio debe realizarse de forma síncrona utilizando RxJava.

2. Manejo Local y Sincronización con Firebase:

- Si la respuesta del servicio indica **estructura Correcta**, los datos decodificados (**etiqueta1d**, **latitud**, **longitud**, **observacion**) deben almacenarse en la tabla **backup_local** de la base de datos SQLite del dispositivo.
- Por cada **5 lecturas correctas** almacenadas localmente (es decir, después de la 5ta, 10ma, 15ta, etc., lectura correcta), se debe sincronizar
- La sincronización consiste en enviar a la colección **backup** en Firebase un nuevo documento que contenga:
 - Un JSON con toda la data actualmente almacenada en **backup_local**.
 - El serial del dispositivo que realiza la operación.
 - La fecha actual en formato String "año-mes-día".
 - Si y solo si el usuario cierra sesión el backup deberá eliminarse
 - lo que significa que si el usuario no cierra sesión y se borran datos localmente cuando, se loguee de nuevo deberá validar si este posee backup y si tiene pintarlo en pantalla

3. Visualización de Datos en RecyclerView:

- El **RecyclerView** en la pantalla principal debe mostrar los datos almacenados en la tabla **backup_local** de la base de datos SQLite.
- Cada elemento del **RecyclerView** debe ser una "tarjeta" que muestre:
 - "etiqueta1d: [valor de la etiqueta1d]"
 - Al lado de la etiqueta1d, un icono de "Mapa" (siguiendo el estilo de iconografía).
 - Debajo de la etiqueta1d y el icono de mapa, un título "Observación:" seguido del valor de la observación.
- La lista debe actualizarse automáticamente cada vez que se almacena un nuevo elemento correctamente.

4. Funcionalidad de Mapa:

- Al presionar el icono de "Mapa" en una tarjeta del **RecyclerView**, se debe navegar a una nueva pantalla.
- Esta nueva pantalla debe estar en **formato horizontal**.
- Debe mostrar un mapa (puedes elegir la librería de mapas que prefieras, ej. Google Maps SDK).

- En el mapa, debes pintar un marcador en la posición geográfica (**latitud**, **longitud**) obtenida del elemento de la lista seleccionado. El icono del marcador debe, si es posible, seguir el estilo de iconografía general.
- Adicionalmente, debes pintar una línea en el mapa desde la posición actual del usuario (debes obtener la ubicación del dispositivo) hasta la posición del marcador. El color de la línea debe ser consistente con la paleta de colores de la aplicación (ej. algún tono de azul).
- Esta pantalla debe tener un botón de "Regresar" que navegue de vuelta a la pantalla principal con el listado.

5. Campo de Texto - Entrada Manual:

- El campo de texto en la pantalla principal no solo es una ayuda visual al escanear, sino que también permite la entrada manual.
- Si el usuario ingresa texto con la estructura esperada (**etiqueta1d-latitud-longitud-observacion**) en el campo de texto y de alguna manera (define cómo, ej. al presionar "Enter" o un botón adicional), se debe convertir este texto a la estructura Base64 esperada por la API (**etiqueta1d:...-latitud:...-longitud:...-observacion:...**) y luego enviar a la API como si se hubiera leído de un QR. Sigue el mismo flujo de validación y almacenamiento local/sincronización.

6. Arquitectura, Inyección de Dependencias y Diseño:

- Debes implementar la aplicación utilizando el patrón de arquitectura MVVM o MVP. Justifica brevemente tu elección.
- Debes utilizar Dagger 2 para la inyección de dependencias en al menos ViewModels/Presenters y cualquier otra dependencia relevante (ej. instancias de API service, base de datos).
- El diseño de las interfaces de usuario debe hacerse utilizando Layout XML, **aplicando la guía de estilo de colores (tonos de azul, blanco, negro/gris oscuro, color de acento) e iconografía limpia descrita anteriormente.**

7. Pruebas:

- Debes incluir un mínimo de **2 pruebas** unitarias o de integración que cubran lógica importante de la aplicación (ej. pruebas para ViewModels/Presenters, pruebas para repositorios que manejen la lógica de negocio, pruebas para la conversión a Base64, etc.). Justifica qué aspectos cubren tus pruebas.

8. CI/CD con GitHub Actions y Firebase Distribution:

- Configura un flujo de trabajo en GitHub Actions (**.github/workflows**) que realice lo siguiente:
 - Compile la aplicación.
 - Ejecute las pruebas que has implementado.
 - Si la compilación y las pruebas son exitosas, despliegue un *debug build* de la aplicación en Firebase App Distribution. **Este despliegue debe dirigirse al proyecto de Firebase que tu creaste, que es donde se encuentra la base de datos de Firestore y desde donde los testers descargarán el APK.**

- Deberás invitar al usuario de GitHub <https://github.com/ingoskr10> al repositorio.
 - Deberás invitar como testers en Firebase App Distribution a los siguientes correos electrónicos:
 - oscart@coordinadora.com
 - sdhajan@coordinadora.com
 - camilov@coordinadora.com
 - **Estos tres correos electrónicos (oscart@coordinadora.com, sdhajan@coordinadora.com, camilov@coordinadora.com) también deben ser asociados al repositorio de GitHub que crees para permitir la revisión del código.** Asegúrate de configurar las credenciales de Firebase de forma segura en los Secrets de GitHub Actions.
-

Consideraciones Adicionales:

- Manejo de permisos de la aplicación (Cámara, Ubicación si decides obtener la posición actual del usuario).
 - Optimización del [RecyclerView](#) para manejar listas grandes (si bien la prueba no especifica un tamaño, es una buena práctica a considerar).
 - Manejo de estados de carga y errores en la UI.
 - Claridad y organización del código (paquetes, nombres de clases, etc.).
 - Asegúrate de que la interfaz de usuario refleje la paleta de colores azules predominante, el uso de texto en negro/gris oscuro/blanco, un color de acento para elementos específicos y un estilo de iconografía limpio y moderno, tal como se observa en los ejemplos proporcionados.
-

Criterios de Evaluación (Referencia):

Tu solución será evaluada en base a los siguientes criterios:

- **Funcionalidad:** ¿La aplicación implementa correctamente todos los requisitos descritos?
- **Arquitectura:** ¿Se aplica de forma consistente el patrón MVVM o MVP? ¿Es la arquitectura limpia y modular?
- **RxJava:** ¿Se utiliza RxJava de forma apropiada para manejar las operaciones asíncronas? ¿Se gestionan correctamente las suscripciones para evitar fugas de memoria?
- **Dagger 2:** ¿Se utiliza Dagger 2 correctamente para la inyección de dependencias?
- **Calidad del Código:** ¿Es el código legible, mantenible y sigue buenas prácticas de programación Java y Android?
- **Manejo de Errores:** ¿Se gestionan adecuadamente los posibles errores?
- **Pruebas:** ¿Las pruebas implementadas cubren lógica importante?

- **CI/CD:** ¿Está configurado correctamente el pipeline de GitHub Actions, el despliegue en Firebase Distribution **en el proyecto asociado**, y se han asociado correctamente los correos/usuarios al repositorio?
 - **Diseño UI:** ¿La interfaz de usuario es funcional, cumple con la estructura solicitada y refleja el estilo de colores e iconografía especificado?
-

Entrega:

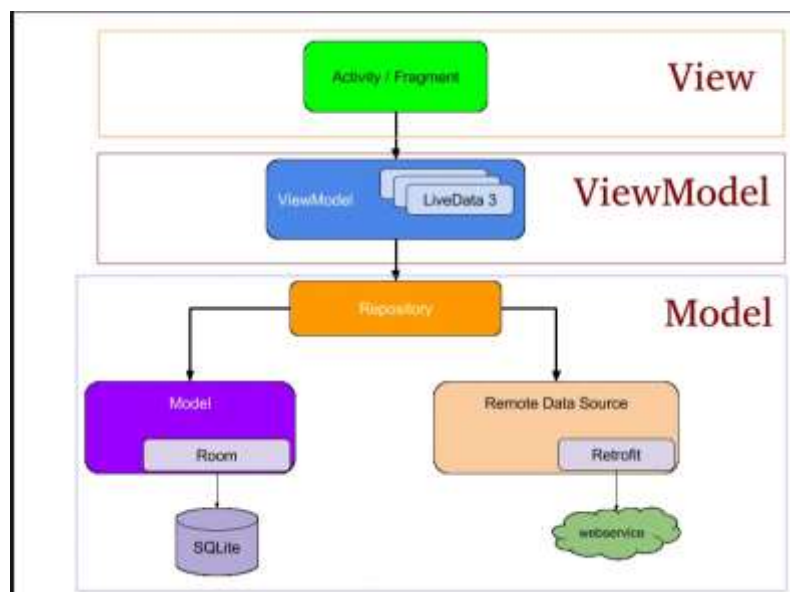
Por favor, proporciona la solución compartiendo el enlace al repositorio de GitHub que contenga el proyecto completo y funcional, incluyendo la configuración de GitHub Actions. Asegúrate de haber invitado al usuario de GitHub y a los correos electrónicos especificados para poder revisar el código y la distribución de Firebase.

¡Mucho éxito!

Apartado de respuestas

Documenta cualquier suposición importante que hayas hecho durante la implementación.

Lo primero fue la arquitectura, al momento de implementar peticiones como yo he utilizado ambas (MVP y MVVM) la más rentable para mí fue MVVM por el tema de modular las peticiones al API y localizar la BD.



En este caso como use Volley vi mejor esta arquitectura para separar la lógica de la BD, para mí, se modulariza de mejor manera con esta arquitectura cuando hacemos peticiones a múltiples tareas asíncronas, iba a usar OkHttp para validar los estados, pero mejor use un JsonObject para manejar el cuerpo de la petición.

- Al momento de meter la base de datos no valide como hacer un cambio de usuario para limpiar la base de datos local, como quería verificar que se podía ingresar a la app sin conexión y ver el mapa con los puntos bien cargados y que se viera, hice la misma lógica que me pedía la prueba pero con diferentes parámetros, porque si cerraba sesión y limpiaba todo no podía comprobar la persistencia, solo matando la app, sin embargo, quería corroborar que se viera cerrando sesión he iniciado, por ende, deje que cuando ingrese un nuevo usuario borre la información del anterior, por esta razón actualice la bd en su versión 2, para implementar el apartado de username.
- Use RequestQueue para comprobar la conexión, con esto hago inyecciones de dependencia al momento de hacer las peticiones http, estas entran en cola con

```
import com.android.volley.RequestQueue;
```

por ende, al momento de hacer una petición de red puedo comprobar la solicitud al api
esto se ve en el paquete

```
package com.svape.gr.coorapp.di.modules;
```

- utilice pruebas unitarias enfocadas en el envío de datos al endpoint, más adelante lo explico

Se debe implementar Volley como petición http, una librería para QR scanning, una librería para mapas, etc.), pero justifica tu elección si no son las más comunes.

Para los QR use esta librería

<https://github.com/yuriy-budiyev/code-scanner>

porque es para Android y esta escrita en Java, entonces al importarlo en mi proyecto es más sencillo, además tiene 1 k de estrellas

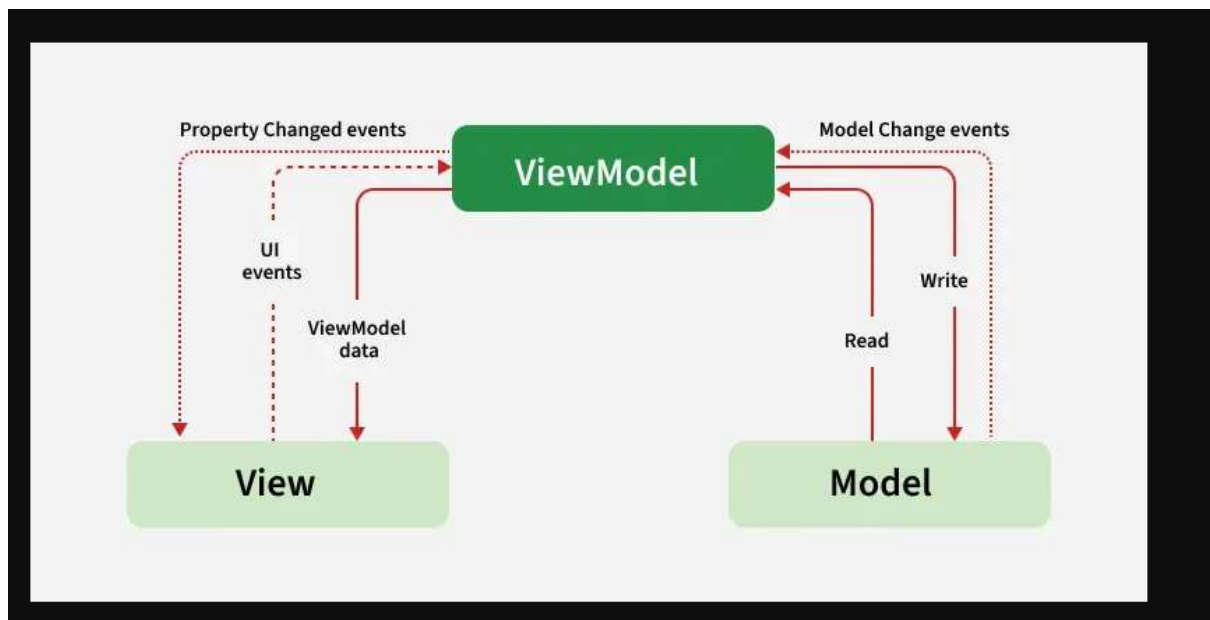
Para el Mapa use Mapbox

<https://www.mapbox.com/>

Trabaje con el durante un año para una empresa que daba un servicio para Ecopetrol enfocada en pozos petroleros, entonces para mí fue la opción más optima.

Debes implementar la aplicación utilizando el patrón de arquitectura MVVM o MVP. Justifica brevemente tu elección.

Al momento de mirar las peticiones a los servicios de manera asíncrona se me hizo mas optimo MVVM, porque no necesito que los componentes tengan tareas asignadas que no le correspondan, solo tienen que recibir lo que necesitan para funcionar, además el databinding es muy útil para referenciar xml.



Debes incluir un mínimo de 2 pruebas unitarias o de integración que cubran lógica importante de la aplicación (ej. pruebas para ViewModels/Presenters, pruebas para repositorios que manejen la lógica de negocio, pruebas para la conversión a Base64, etc.). Justifica qué aspectos cubren tus pruebas.

En las pruebas unitarias incluí:

- El tema de las peticiones al API para comprobar escenarios de éxito, error, simulación de peticiones y manejo asíncrono con RxJava para las operaciones.
- En el viewmodel valido el formato de los datos y la fecha con reglas de manera síncrona
- En el DataParser valido datos vacíos, entradas vacías, validación de formato y datos.